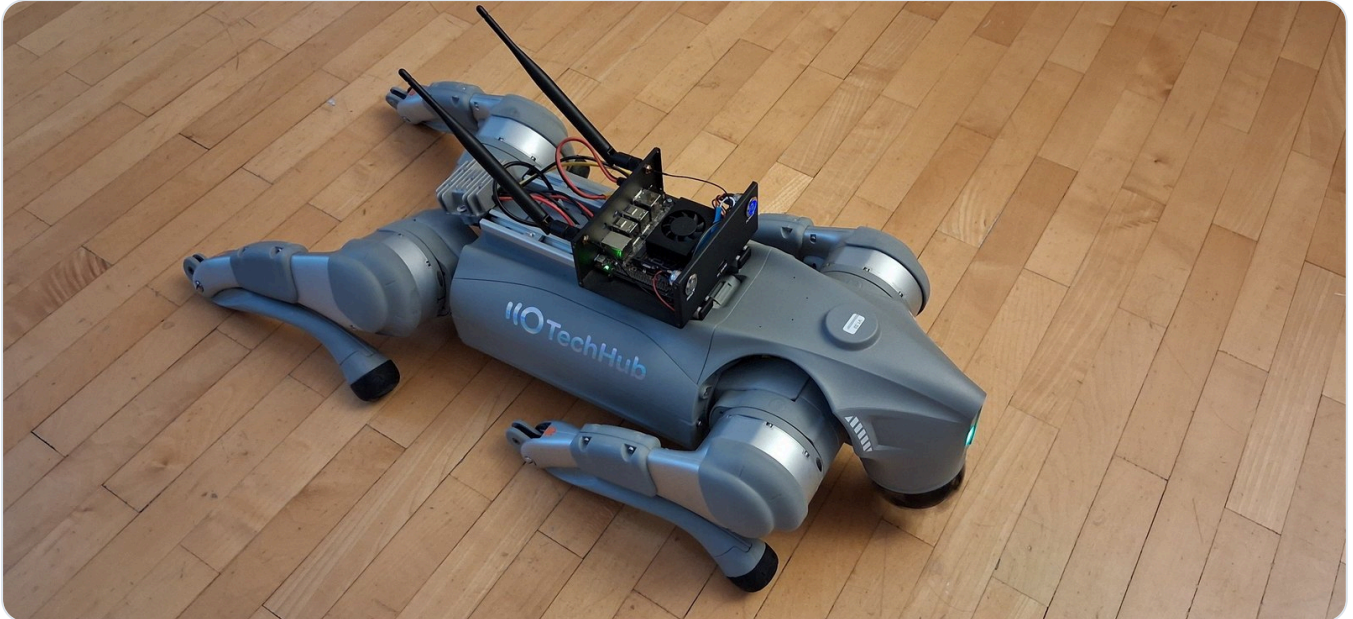


Go2 Robot Integration Guide

Connect the robot to your smart home: read its state, send it places, and view its camera over plain network interfaces.



The unit. A Unitree Go2 with a Jetson Xavier NX2 mounted on its back. The Jetson runs ROS 2 and talks to the robot's internal controller over DDS, exposing the two network interfaces used below.

Home Assistant

Foxglove WebSocket

HTTP camera

Nav2 goals

The robot exposes two network interfaces you can build against. We run the map and navigation; your side sends goals and reads state.

Interface	Address	Use it for
Foxglove WebSocket bridge	<code>ws://robot.local:8765</code>	Read any sensor topic and send goals, from any language.
Camera HTTP server	<code>http://robot.local:8080</code>	Grab camera frames over plain HTTP.

Connect

- Join the same WiFi as the robot. Use 5 GHz for smooth video; commands and sensor data are tiny and work on any band.
- `robot.local` below is a placeholder. Use the robot's actual IP address (we give it to you on the day).
- You also get a short list of `(x, y, yaw)` coordinates for useful spots on the map.

Camera

The easiest thing to wire up. The robot serves its latest frame as a JPEG over HTTP, no ROS needed:

- `GET http://robot.local:8080/latest.jpg` returns the newest frame. Poll it for a ~10 fps feed.
- `GET http://robot.local:8080/health` returns stream status as JSON.

BASH

```
curl -o frame.jpg http://robot.local:8080/latest.jpg
```

Read state and send goals

Everything else goes over the Foxglove WebSocket bridge at `ws://robot.local:8765`. Subscribe to any sensor topic and publish navigation goals. On connect, offer both subprotocols `foxglove.websocket.v1` and `foxglove.sdk.v1` and let the server pick.

Quickest test: Foxglove Studio (no code)

1. Open Foxglove (`app.foxglove.dev` or the desktop app), choose **Foxglove WebSocket**, enter `ws://robot.local:8765`, Open.
2. Add panels and pick topics like `/odom`, `/battery`, `/camera/compressed` to watch the robot.
3. To drive it, add a Publish panel on `/goal_pose` (schema `geometry_msgs/msg/PoseStamped`), set `frame_id = "map"`, `position.{x,y}`, `orientation.w = 1`, Publish.

Reading topics from Python

A plain WebSocket client plus rclpy's deserializer reads topics cleanly, no DDS or ROS networking required:

PYTHON

```
# pip install websocket-client · rclpy ships with ROS 2
import json, struct, websocket
from rclpy.serialization import deserialize_message
from rosidl_runtime_py.utilities import get_message

ws = websocket.create_connection(
    "ws://robot.local:8765",
    subprotocols=["foxglove.websocket.v1", "foxglove.sdk.v1"])

# Server sends JSON "advertise" frames listing every topic; collect them.
channels = {}
ws.settimeout(2.0)
try:
    while True:
        frame = ws.recv()
        if isinstance(frame, str):
            m = json.loads(frame)
            if m.get("op") == "advertise":
                for ch in m["channels"]:
                    channels[ch["id"]] = ch
except websocket.WebSocketTimeoutException:
    pass

# Subscribe to one topic and decode its CDR messages.
cid = next(c["id"] for c in channels.values() if c["topic"] == "/odom")
ws.send(json.dumps({"op": "subscribe", "subscriptions": [{"id": 1, "channelId": cid}]}))
MsgType = get_message(channels[cid]["schemaName"]) # e.g. nav_msgs/msg/Odometry

ws.settimeout(None)
while True:
    frame = ws.recv()
    if isinstance(frame, bytes) and frame and frame[0] == 0x01: # binary message frame
        msg = deserialize_message(frame[13:], MsgType) # skip 13-byte header
        print(msg.pose.pose.position.x)
```

Sending a goal

Publish a `geometry_msgs/PoseStamped` on `/goal_pose` with `frame_id "map"` and your `(x, y)`. Easiest from the Studio Publish panel; from code, advertise a CDR channel and send the message (the official JS client [@foxglove/ws-protocol](#) does this cleanly). Publish again any time to redirect.

Topics you'll use

Topic	Type	Direction	Notes
<code>/goal_pose</code>	<code>geometry_msgs/PoseStamped</code>	publish	Navigation goal, <code>frame_id: "map"</code> , meters.
<code>/odom</code>	<code>nav_msgs/Odometry</code>	subscribe	Live position and velocity.
<code>/battery</code>	<code>sensor_msgs/BatteryState</code>	subscribe	Battery percentage.
<code>/camera/compressed</code>	<code>sensor_msgs/CompressedImage</code>	subscribe	JPEG, ~10 Hz (or use the HTTP camera above).

Home Assistant as a middle layer

For a smart home, put **Home Assistant (HA)** in the middle so dashboards, automations and voice all drive the robot through one layer, alongside your other devices.

Why it fits: the camera drops straight in, and the map coordinates map onto buttons ("Go to entrance"). HA is great for high-level goals, monitoring and automation, **not** for tight real-time teleop.

1 · Camera, drops in directly

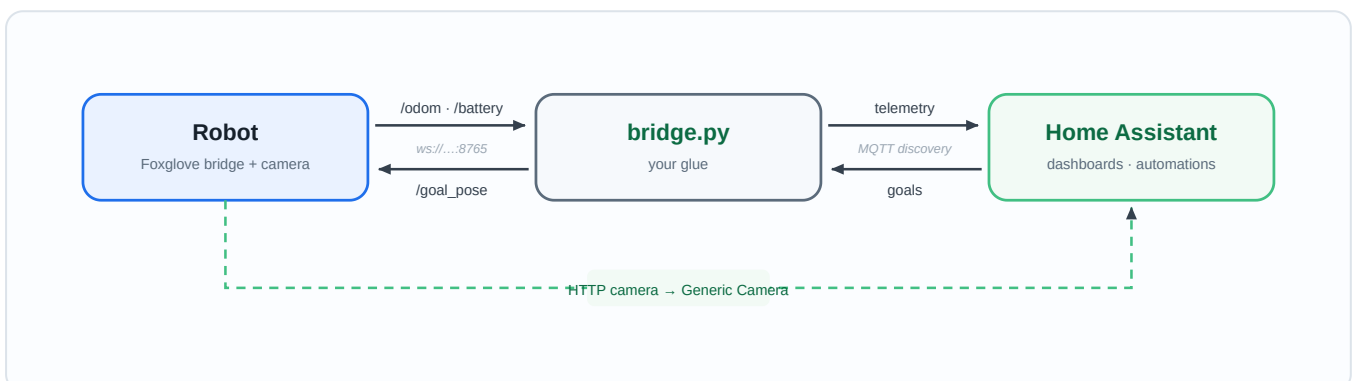
The HTTP camera maps onto HA's **Generic Camera** integration. Point it at the still-image URL for a camera entity, no code:

```
YAML · CONFIGURATION.YAML

camera:
  - platform: generic
    name: Go2 Camera
    still_image_url: "http://<robot-ip>:8080/latest.jpg"
    framerate: 10
    verify_ssl: false
```

2 · State and goals, a small MQTT bridge

HA doesn't speak the Foxglove protocol, so bridge it to **MQTT**, which HA supports natively. A tiny script reuses the Python reader above: it republishes `/odom` and `/battery` to MQTT, and turns incoming goals into a `/goal_pose` publish.



PYTHON · BRIDGE.PY (SKETCH)

```
import json, paho.mqtt.client as mqtt

mq = mqtt.Client()
mq.connect("homeassistant.local", 1883)

# Announce a battery sensor to HA once, via MQTT discovery:
mq.publish("homeassistant/sensor/go2_battery/config", json.dumps({
    "name": "Go2 Battery", "state_topic": "go2/battery",
    "unit_of_measurement": "%", "device_class": "battery",
    "unique_id": "go2_battery"}), retain=True)

# In your Foxglove read loop, forward values to MQTT:
mq.publish("go2/battery", round(battery_pct))

# And when HA sends {"x":..., "y":...}, publish a PoseStamped on /goal_pose.
```

HA then gives you a battery sensor, a position sensor, and `button` / `script` entities that send goals at your named coordinates. From there it's normal HA: dashboards, automations ("if battery < 20 %, send home"), schedules and voice.

Already on ROS 2?

If your machine is on the robot's network and you'd rather skip the bridge, match the robot's `ROS_DOMAIN_ID` and set `ROS_AUTOMATIC_DISCOVERY_RANGE=SUBNET`; then `/goal_pose`, `/odom` and the rest show up as normal ROS 2 topics.

Good to know

- The robot looks after itself: speed is capped, and it stops if commands stop arriving. It can be e-stopped at any time.
- Goals are followed whenever our navigation is running.
- Goals land on our map, so accuracy follows our map and localization.

References

- **Foxglove:** Studio at app.foxglove.dev, docs at foxglove.dev/docs; JS client @foxglove/ws-protocol
- **Home Assistant:** Generic Camera at home-assistant.io/integrations/generic; MQTT and discovery at home-assistant.io/integrations/mqtt
- **ROS 2 message types:** `geometry_msgs/PoseStamped`, `nav_msgs/Odometry`, `sensor_msgs/BatteryState`, `sensor_msgs/CompressedImage`
- **Nav2** (navigation behind `/goal_pose`): docs.nav2.org